Repaso del día 5

16 preguntas sobre Etherscan, Smart Contracts, Solidity, gas, eventos, estándares ERC y seguridad

Las 16 preguntas — Repaso día 5

1. ¿Qué problema resuelve un explorador como Etherscan? ¿Para qué lo usarías tú en el día a día?
2. Si alguien te dice 'te he enviado Ethers' pero todavía no aparece en tu wallet, ¿cómo podrías comprobarlo tú mismo sin fiarte de su palabra?
3. Una transacción puede aparecer minada en Etherscan y aun así estar marcada como 'Failed'. ¿Cómo es posible? ¿Quién paga el gas en ese caso?
4. ¿En qué se parece y en qué se diferencia una dirección de smart contract de la dirección de tu wallet?
5. ¿Qué garantía te da que un Smart Contract esté 'verified' en Etherscan? ¿Te garantiza que es seguro?
6. ¿Por qué se llama a Ethereum 'el computador mundial'? ¿Qué tiene de especial frente a, por ejemplo, un servidor de Amazon?
7. ¿Por qué un Smart Contract no puede usar números aleatorios o llamadas a internet?
8. ¿Por qué consultar tu saldo de USDC es gratis pero enviar 0.001 ETH a alguien cuesta dinero? Desde fuera parece la misma operación.

9. El contrato Counter solo tiene una variable y tres funciones, pero permite entender algo profundo de la blockchain. ¿Qué es lo importante que enseña?

10. ¿Por qué guardar un dato en blockchain es muchísimo más caro que sumarlo o calcularlo? ¿Qué implica esto para el diseño de un contrato?

11. Si todos los datos ya están en la blockchain, ¿para qué necesitamos que un contrato 'emita eventos’?

12. Si quisieras tokenizar la propiedad de tu coche, ¿usarías un ERC-20 o un ERC-721? ¿Y si quisieras emitir tu propia moneda?

13. ¿Por qué a la composabilidad de los Smart Contracts se le llama 'Lego Money'? ¿Qué se puede hacer ahora que antes no?

14. Si un Smart Contract es inmutable una vez desplegado, ¿cómo es posible que Uniswap o Aave hayan ido cambiando su código a lo largo de los años?

15. Si un Smart Contract no puede generar números aleatorios, ¿cómo se las arregla un juego en blockchain para sortear un premio o un mint aleatorio de NFTs?

16. El hack de The DAO en 2016 dividió a la comunidad Ethereum en dos. ¿Qué dilema moral planteó? ¿Por qué fue tan difícil decidir?

PREGUNTA 1 / 16

¿Para qué sirve un explorador?

Pregunta

¿Qué problema resuelve un explorador como Etherscan? ¿Para qué lo usarías tú en el día a día?

Respuesta razonada

La información de la blockchain es pública pero ilegible para un humano: son bytes y hashes. Etherscan traduce esa información en algo que puedes leer y navegar — transacciones, saldos, contratos, eventos. Lo usas para comprobar si una transacción se completó, ver si un wallet recibió tus fondos, auditar el código de un contrato antes de interactuar con él, o investigar qué está haciendo un proyecto. Es la 'ventana al libro mayor'.

PREGUNTA 2 / 16

Confiar sin confiar

Pregunta

Si alguien te dice 'te he enviado dinero' pero todavía no aparece en tu wallet, ¿cómo podrías comprobarlo tú mismo sin fiarte de su palabra ni del banco?

Respuesta razonada

Ese es uno de los superpoderes de blockchain: cualquiera puede verificar cualquier cosa por su cuenta. Le pides el hash de la transacción, lo buscas en Etherscan y ves todo: cuándo se envió, de qué wallet, a qué wallet, cuánto, si está confirmada o aún pendiente. No necesitas un intermediario que te lo confirme. Es la materialización del 'don't trust, verify'.

PREGUNTA 3 / 16

Transacciones que fallan

Pregunta

Una transacción puede aparecer minada en Etherscan y aun así estar marcada como 'Failed'. ¿Cómo es posible? ¿Quién paga el gas en ese caso?

Respuesta razonada

Una transacción se mina si el validador la incluye en un bloque, pero su ejecución puede fallar (por ejemplo, un require que no se cumple en el contrato, o intentas vender más tokens de los que tienes). El cambio se revierte: el estado queda como antes. Pero el gas YA se ha consumido — lo paga igualmente quien firmó la transacción. Los validadores no trabajan gratis: aunque tu lógica falle, su esfuerzo computacional sí cobra.

PREGUNTA 4 / 16

Cuentas vs Contratos

Pregunta

¿En qué se parece y en qué se diferencia una dirección de smart contract de la dirección de tu wallet?

Respuesta razonada

Se parecen en que ambas son direcciones de 42 caracteres y pueden recibir ETH y tokens. Se diferencian en quién manda: tu wallet la controla una persona con su clave privada (puede firmar y decidir cuándo enviar). Una dirección de contrato la controla el código que tiene desplegado: no firma nada por su cuenta, solo reacciona cuando alguien lo invoca. Tu wallet es una persona; el contrato es un programa autónomo.

PREGUNTA 5 / 16

¿Por qué auditar?

Pregunta

¿Qué garantía te da que un Smart Contract esté 'verified' en Etherscan? ¿Te garantiza que es seguro?

Respuesta razonada

Solo te garantiza una cosa: que puedes leer el código fuente del contrato y comprobar exactamente lo que hace. NO te garantiza que sea seguro — puede tener bugs catastróficos perfectamente visibles. Verified es transparencia, no seguridad. Pero es el requisito mínimo: sin verificar, fiarte de un contrato es fiarte a ciegas. Con el código visible, al menos puedes auditarlo tú, leer auditorías de terceros o que la comunidad lo revise.

PREGUNTA 6 / 16

¿Por qué 'computador mundial'?

Pregunta

¿Por qué se llama a Ethereum 'el computador mundial'? ¿Qué tiene de especial frente a, por ejemplo, un servidor de Amazon?

Respuesta razonada

Porque ejecuta código que nadie puede parar, censurar ni modificar. Un servidor de Amazon depende de Amazon: si quiebran, si la justicia bloquea la cuenta o si deciden apagarlo, tu programa muere. Ethereum lo ejecutan miles de nodos en todo el mundo simultáneamente, y todos llegan al mismo resultado. Para 'apagar' un smart contract tendrías que apagar la red entera. Es lento y caro comparado con AWS, pero a cambio te da confianza sin necesitar confiar en nadie.

PREGUNTA 7 / 16

Determinismo

Pregunta

¿Por qué un Smart Contract no puede usar la hora del sistema, números aleatorios o llamadas a internet?

Respuesta razonada

Porque el resultado del contrato debe ser idéntico en TODOS los nodos del mundo. Si un nodo en Tokio consulta la hora y otro en Madrid también, obtendrán valores distintos. Si llaman a una API externa, podrían recibir respuestas diferentes. Sin determinismo no hay consenso, y sin consenso no hay blockchain. Por eso los contratos solo pueden usar datos que estén YA en la cadena, idénticos para todos. Para lo demás existen los oráculos.

PREGUNTA 8 / 16

¿Por qué cuesta dinero?

Pregunta

¿Por qué consultar tu saldo de USDC es gratis pero enviar 0.001 ETH a alguien cuesta dinero? Desde fuera parece la misma operación.

Respuesta razonada

Porque cambia QUIÉN lo ejecuta. Consultar es pedirle a un nodo: 'oye, dime cuál es este valor'. Solo ese nodo responde, no hay consenso ni bloque, no se cobra nada. Enviar es decirle a la red entera: 'todos vosotros aceptad que este cambio es válido, lo incluís en un bloque y lo replicáis para siempre'. Eso exige trabajo coordinado de todos los validadores y un cambio permanente en el estado mundial. Por eso lo segundo cuesta y lo primero no.

PREGUNTA 9 / 16

El Counter

Pregunta

El contrato Counter solo tiene una variable y tres funciones, pero permite entender algo profundo de la blockchain. ¿Qué es lo importante que enseña?

Respuesta razonada

Que el estado del contrato NO vive en tu ordenador, vive en la blockchain. Si tú haces increment(), no estás cambiando algo tuyo: estás cambiando un valor que VEN todos los demás nodos del mundo. Cuando alguien al otro lado del planeta consulte el contrato, verá tu cambio. Esa es la magia: estado global compartido y consistente, sin que ninguna empresa lo gestione. Lo demás (variables, funciones, require) son detalles sintácticos.

PREGUNTA 10 / 16

El precio del recuerdo

Pregunta

¿Por qué guardar un dato en blockchain es muchísimo más caro que sumarlo o calcularlo? ¿Qué implica esto para el diseño de un contrato?

Respuesta razonada

Porque cuando un contrato guarda algo en storage, ese dato se replica en TODOS los nodos del mundo, para siempre, y ocupa espacio en disco eternamente. Calcular algo en RAM y olvidarlo no cuesta apenas nada; grabarlo en una base de datos global e inmutable cuesta mucho. Implicación clave: los buenos contratos guardan lo MÍNIMO en la cadena. Lo demás lo emiten como eventos (más baratos) o lo dejan fuera de la cadena. 'Guardar es caro' es la primera lección de un buen desarrollador de Solidity.

PREGUNTA 11 / 16

¿Por qué emitir eventos?

Pregunta

Si todos los datos ya están en la blockchain, ¿para qué necesitamos que un contrato 'emita eventos'?

Respuesta razonada

Porque rastrear lo que ha pasado revisando todos los bloques es carísimo y lento. Si quieres saber 'todas las veces que se ha transferido USDC en el último año', tendrías que leer millones de transacciones e interpretarlas. Los eventos son la forma estructurada que tiene el contrato de gritar 'oye, ha pasado esto': se graban en logs indexados y las apps pueden suscribirse a ellos en tiempo real. Etherscan, MetaMask, OpenSea, todos viven de leer eventos.

PREGUNTA 12 / 16

Eligiendo estándar

Pregunta

Si quisieras tokenizar la propiedad de tu coche, ¿usarías un ERC-20 o un ERC-721? ¿Y si quisieras emitir tu propia moneda?

Respuesta razonada

El coche es un ERC-721 (NFT): es un activo ÚNICO, con número de bastidor, kilómetros y desperfectos específicos. No es intercambiable con otro coche aunque sea el mismo modelo. Una moneda, en cambio, es un ERC-20: una unidad es indistinguible de otra y son intercambiables 1:1. La clave es la pregunta '¿esta unidad es idéntica a esa otra?': si sí, ERC-20 (fungible); si no, ERC-721 (no fungible).

PREGUNTA 13 / 16

Lego Money

Pregunta

¿Por qué a la composabilidad de los Smart Contracts se le llama 'Lego Money'? ¿Qué se puede hacer ahora que antes no?

Respuesta razonada

Porque cualquier contrato puede llamar a otro sin pedir permiso, como piezas de Lego que encajan entre sí. Yo puedo construir un producto que use Aave para pedir prestado, Uniswap para intercambiar y Chainlink para precios — todo en una sola transacción y sin tener que firmar acuerdos con ninguna de esas empresas. Antes, integrar tres bancos exigía meses de negociación; aquí son tres líneas de código. Eso multiplicó la innovación financiera de forma brutal. También multiplicó los hacks: cada conexión es una superficie de ataque.

PREGUNTA 14 / 16

Inmutabilidad real

Pregunta

Si un Smart Contract es inmutable una vez desplegado, ¿cómo es posible que Uniswap o Aave hayan ido cambiando su código a lo largo de los años?

Respuesta razonada

Hacen una trampa elegante con un patrón llamado 'proxy'. Hay dos contratos: uno público (la 'fachada') al que llaman los usuarios, y otro detrás con la lógica real. La fachada no tiene lógica, solo redirige las llamadas al contrato de detrás. Para 'actualizar' Uniswap, despliegan un contrato nuevo y le dicen a la fachada que apunte a él. Para los usuarios la dirección no cambia, pero la lógica sí. Así que 'inmutable' es más una elección de diseño que una propiedad automática.

PREGUNTA 15 / 16

Aleatoriedad sin aleatoriedad

Pregunta

Si un Smart Contract no puede generar números aleatorios, ¿cómo se las arregla un juego en blockchain para sortear un premio o un mint aleatorio de NFTs?

Respuesta razonada

Usando un oráculo: un servicio externo (típicamente Chainlink VRF) que genera el número aleatorio y lo introduce en la blockchain con una prueba criptográfica que demuestra que es genuino y no manipulado. El contrato no genera el número, lo PIDE a un actor externo y verifica que la respuesta es válida. La aleatoriedad sigue viniendo de fuera, pero gracias a la prueba, todos los nodos confían en ella aunque ninguno la generó.

PREGUNTA 16 / 16

Code is law... ¿o no?

Pregunta

El hack de The DAO en 2016 dividió a la comunidad Ethereum en dos. ¿Qué dilema moral planteó? ¿Por qué fue tan difícil decidir?

Respuesta razonada

Un atacante robó 60M USD aprovechando un bug del contrato. El robo era 'legal' según el código: las reglas del contrato se cumplían exactamente. Esto enfrentó dos principios: 'code is law' (el código es la única ley, hay que aceptar el robo) versus 'justicia material' (es un robo aunque sea técnicamente válido, hay que revertirlo). La comunidad eligió revertirlo creando un hard-fork (Ethereum), pero parte de la comunidad rechazó esa decisión y siguió en la cadena original (Ethereum Classic). Fue la pérdida de la inocencia: la inmutabilidad absoluta es un ideal, no una garantía.

Blockchain

Práctica de Smart Contracts

CÓDIGO

CounterWL.sol — Contador con Whitelist

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;
contract Counter {
   uint public count;
   address public owner;
   mapping(address => bool) public whitelist;
   constructor() {
      owner = msg.sender;
   }
   modifier onlyOwner() {
      require(msg.sender == owner, "Solo el owner puede hacer esto");
      _;
   }
   modifier onlyWhitelisted() {
      require(whitelist[msg.sender], "No estas en la whitelist");
      _;
   }
   function addToWhitelist(address user) external onlyOwner {
      whitelist[user] = true;
   }
   function removeFromWhitelist(address user) external onlyOwner {
      whitelist[user] = false;
   }
   function increment() external onlyWhitelisted {
      count += 1;
   }
   function decrement() external {
      require(count > 0, "No puede ser negativo");
      count -= 1;
   }
   function reset() external {
      count = 0;
   }
}
```

CÓDIGO

Vault.sol — qué vamos a desplegar

Caja fuerte temporal: bloquea ETH durante un tiempo

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;

contract Vault {
    mapping(address => uint) public deposits;       // saldo bloqueado por usuario
    mapping(address => uint) public unlockTime;     // timestamp de desbloqueo

    event Deposited(address indexed user, uint amount, uint unlockAt);
    event Withdrawn(address indexed user, uint amount);

    function deposit(uint lockSeconds) external payable {
        require(msg.value > 0, "Envia ETH");
        deposits[msg.sender] += msg.value;
        unlockTime[msg.sender] = block.timestamp + lockSeconds;
        emit Deposited(msg.sender, msg.value, unlockTime[msg.sender]);
    }

    function withdraw() external {
        require(block.timestamp >= unlockTime[msg.sender], "Aun bloqueado");
        uint amount = deposits[msg.sender];
        deposits[msg.sender] = 0;                   // efecto ANTES de la interacción
        payable(msg.sender).transfer(amount);
        emit Withdrawn(msg.sender, amount);
    }
}
```

Repasar el patrón Checks-Effects-Interactions (línea naranja): primero comprobar, luego modificar estado, luego interactuar. Es lo que evita reentrancy.

PRÁCTICA 2

Desplegar nuestro propio Vault

Pasos en Remix + Sepolia

1. Acceder a Remix: `https://remix.ethereum.org/`
2. Borrar los contratos de ejemplo. Crear nuevo fichero `Vault.sol` y pegar el código.
3. Compilar (icono Solidity Compiler). Verificar que no hay errores. Ver el bytecode generado.
4. En la pestaña Deploy: **Environment → Injected Provider (MetaMask)**. Comprobar que estás en Sepolia.
5. Pulsar Deploy. MetaMask te pide firmar la transacción. Confirmar y esperar a que se incluya en un bloque.
6. Probar `deposit(60)` con 0.001 ETH. Intentar `withdraw()` inmediatamente — debe fallar ("Aun bloqueado").
7. Esperar 60 segundos. Volver a intentar `withdraw()` — ahora funciona. Verificar el balance recuperado.
8. Buscar la dirección del contrato en sepolia.etherscan.io y ver tus transacciones + eventos emitidos.

PRÁCTICA 3

Reto: añadir un fee de retirada

Aplicar control de acceso al Vault

Modifica el contrato Vault para que cobre un 1% de fee al retirar, que se acumula en una variable y solo el owner puede reclamar.

Pistas:

1. Añadir variable `address public owner` y asignarla en el constructor.
2. Añadir variable `uint public feesAcumulados`.
3. En withdraw: calcular fee (amount × 1 / 100), restarlo del envío y sumarlo a feesAcumulados.
4. Añadir un modifier `onlyOwner`.
5. Crear función `claimFees() external onlyOwner` que envía feesAcumulados al owner.

Probar: desde una segunda cuenta de MetaMask, intentar invocar `claimFees()`. Debe fallar con "No autorizado".

REALIDAD

¿Cuánto cuesta desplegar este contrato?

Estimación orientativa

Red	Gas estimado	Coste despliegue	Coste invocación
Ethereum mainnet	~600.000	**~10-50 USD**	~1-5 USD por tx
Sepolia (testnet)	~600.000	**0 USD (ETH gratis)**	0 USD
Polygon	~600.000	**~0.05 USD**	~0.001 USD
Arbitrum / Optimism	~600.000	**~0.5-2 USD**	~0.05 USD

La elección de red importa: *el mismo contrato puede costar 50 USD en mainnet o 0.05 USD en Polygon. Por eso muchos proyectos se despliegan en L2 o sidechains. Y por eso en clase usamos siempre Sepolia.*

PUENTE

Smart Contracts en EVM vs Chaincodes en Fabric

Lo que has aprendido aquí (Solidity, EVM, gas, deploy) es el modelo de Ethereum. Hyperledger Fabric tiene un modelo distinto: chaincodes. Mismas ideas (lógica de negocio en la blockchain), implementación radicalmente distinta.

Aspecto	EVM (Ethereum y similares)	Hyperledger Fabric
Lenguajes	Solidity, Vyper, Yul	Go, JavaScript/TypeScript, Java
Despliegue	1 transacción	Lifecycle: package → install → approve (cada org) → commit
Estado	Storage slots (256 bits)	World State (key-value en LevelDB o JSON en CouchDB)
Pago de gas	Sí, en ETH	No (red permisionada, sin moneda nativa)
Ejecución	Order → Execute (todos los nodos)	Endorse → Order → Validate (solo endorsing peers)
Identidad	Wallet pseudónima (0x...)	Certificado X.509 emitido por la CA de la org

Debate – Desplegar Caja Fuerte

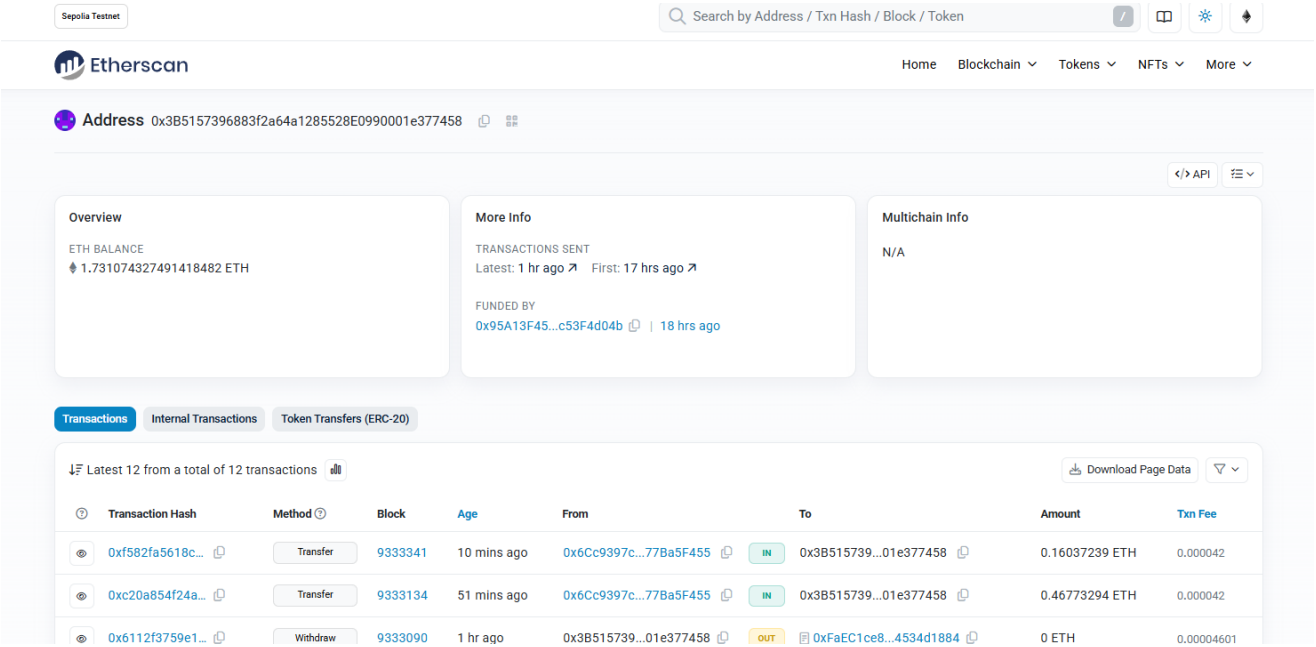
Experiencia técnica y de usuario

- ¿Qué te ha parecido el uso de **Metamask** para conectarte a **Remix**?
- ¿Es **intuitivo o complejo** el proceso de firmar transacciones?
- ¿Qué riesgos ves en que el usuario tenga que **aprobar cada interacción manualmente**?
- ¿Podría un contrato **malicioso** aprovechar una firma descuidada?
- ¿Qué aprendiste sobre la relación entre **frontend (Remix)** y **backend (Blockchain)**?

Accesibilidad y permanencia

- Un contrato desplegado es **visible y accesible por cualquiera**: ¿qué ventajas e inconvenientes tiene?
- Si cualquiera puede **invocar tu contrato**, ¿cómo se garantiza la seguridad y el control de uso?
- ¿Qué implica el principio **“deploy once, consume all”** en términos de **propiedad intelectual** y **responsabilidad**?
- ¿Qué tan difícil sería **ocultar información sensible** en un contrato público?
- ¿Debería existir un modo de **retirar o modificar** un contrato una vez desplegado?

Práctica – Usar Etherscan

Entrar en https://sepolia.etherscan.io/

- Ir al Smart Contract de Caja Fuerte y ver todas las interacciones que ha habido